

5 SECURITY HOLES IN YOUR VIBE-CODED APP'S LOGIN SCREEN

(and the exact prompts to fix them)

Claude Code just shipped you a login screen. Feels like a milestone. It's also the easiest surface in your app to break – here's what it's probably missing, and the exact prompt to hand back to your AI coding tool to fix each one.

[01/05]

> Session Token in local Storage

Your auth token sits in localStorage, which means any injected script – one XSS vulnerability – can read it and hijack the session. Tokens like this belong in an httpOnly cookie, invisible to JavaScript entirely.

Fix it – paste this prompt:

```
$ My vibe-coded app stores its auth/session token in localStorage. Migrate me to httpOnly, Secure, SameSite=Lax cookies using [Supabase @supabase/ssr | Clerk]. Show the exact client + server changes, confirm no token is ever readable from window.localStorage, and tell me how to verify it in DevTools.
```

[02/05]

> Admin Check Happens Client-Side

If your role check runs in the browser, it's not a security check – it's a suggestion. Anyone can open DevTools, flip a variable, and grant themselves admin. The check has to be re-verified on the server, every time.

Fix it – paste this prompt:

```
$ Audit every API route / server action in my app for missing server-side authorization. For each protected action, add a check that re-verifies the user's identity and role from the server session (and add a matching Supabase RLS policy). List which routes were unprotected and show the before/after for each.
```

[03/05]

> No Two-Factor Authentication or Email Verification

Without email verification, anyone can sign up as anybody – no proof they own the email they typed. That opens the door to fake accounts and impersonation from day one.

Fix it – paste this prompt:

```
$ Enable required email verification in [Supabase Auth | Clerk] and block unverified users from any write/sensitive action. Show me the provider setting to flip, the server-side guard that checks email_confirmed_at (or Clerk's verified status), and a redirect to a "please verify" screen.
```

[04/05]

> No Rate Limiting on Login / Reset

If login and password-reset endpoints don't throttle attempts, a bot can brute-force them all day – unlimited guesses, no consequence.

Fix it – paste this prompt:

```
$ Add rate limiting and temporary lockout to my login, signup, and password-reset flows. Use [Supabase Auth's built-in rate limits | Upstash Ratelimit middleware]. Specify sensible limits (e.g. 5 attempts / 15 min per IP+email), the lockout behavior, and how to test that the 6th attempt is blocked.
```

[05/05]

> No Password Rules or Leaked-Password Check

No minimum strength, no check against known-breached passwords – "123456" gets accepted just fine. That's an open invitation.

Fix it – paste this prompt:

```
$ Enforce a strong password policy in [Supabase Auth | Clerk]: minimum length 12, block compromised passwords via the HaveIBeenPwned / leaked-password setting, and surface a clear strength error in my sign-up UI. Show the provider setting and the client validation, and confirm a known-breached password is rejected.
```

Ship secure. That's the vibe.

@swapsays_wtf